

MSE 413 Lab 4 Report

Neural Networks and Deep Learning

Name: Tom Dang
Student ID: 301548582

Name: Sam Drubetsky
Student ID: 301452017

Date: 2025/03/30

Background

In this lab, we investigate the structure and application of artificial neural networks (ANNs) and convolutional neural networks (CNNs). In this case, it is the application of detecting tumors and brain disease through a brain tumor MRI dataset.

An artificial neural network is made of neurons, which are similar to the human brain neurons. It is often used in solving complex nonlinear regression and classification problems and detecting all possible interactions between predictor variables. The trade-offs of ANN are the lack of integrity in how it does classification work, great computational cost, and easily overfit during training.

Convolutional neural network focuses on the capability to capture space- and time-related variables. It is better than ANN because it reduces the number of parameters involved and reuses weights. It is more robust to overfitting for image datasets.

Part 1

1.1 Use “keras.preprocessing.image_dataset_from_directory” to upload your images into two separate training and test datasets in your notebook.

```
np.random.seed(42)
tf.random.set_seed(42)

# 1.1 Use keras.preprocessing.image_dataset_from_directory to upload images

train_dir = "Brain_tumor/Training"
test_dir = "Brain_tumor/Testing"

train_dataset = image_dataset_from_directory(
    train_dir,
    image_size=(224, 224), # We'll resize to a common size
    batch_size=32,
    shuffle=True
)

test_dataset = image_dataset_from_directory(
    test_dir,
    image_size=(224, 224),
    batch_size=32
)
```

```
Found 5712 files belonging to 4 classes.
Found 1311 files belonging to 4 classes.
```

Figure 1: Importing image dataset for training and testing

1.2 Report the size of each dataset (number of image data) as well as the name and number of classes.

```
# 1.2 Report the size of each dataset and class information
class_names = train_dataset.class_names
print("Class names:", class_names)
print("Number of classes:", len(class_names))

num_train_images = sum([len(files) for _, _, files in os.walk(train_dir)])
num_test_images = sum([len(files) for _, _, files in os.walk(test_dir)])

print(f"Number of training images: {num_train_images}")
print(f"Number of testing images: {num_test_images}")

for class_name in class_names:
    class_dir = os.path.join(train_dir, class_name)
    num_images = len(os.listdir(class_dir))
    print(f"Class '{class_name}': {num_images} images")
```

```
Class names: ['glioma', 'meningioma', 'notumor', 'pituitary']
Number of classes: 4
Number of training images: 5712
Number of testing images: 1311
Class 'glioma': 1321 images
Class 'meningioma': 1339 images
Class 'notumor': 1595 images
Class 'pituitary': 1457 images
```

Figure 2: Exploration of different types of MRI brain tumor scans

The dataset provides 5712 images in total for 4 different types of MRI scans, including glioma, meningioma, pituitary and no tumor as the base cases for a healthy brain. There are 1321, 1339, and 1457 images for glioma, meningioma, and pituitary, respectively. Overall, the dataset is balanced for training, and there is no first sight of an imbalanced dataset or focusing on a specific case for detection. There are 1311 images in total for testing across 4 cases.

1.3 Report the shape (height, width, channels) of each image

```
# 1.3 Report the shape of each image
for images, labels in train_dataset:
    print(f"Image shape: {images.shape[1:]}")
```

```
Image shape: (224, 224, 3)
Image shape: (224, 224, 3)
Image shape: (224, 224, 3)
Image shape: (224, 224, 3)
Image shape: (224, 224, 3)
```

Figure 3: Exploring the variables that construct the image (RGB)

3 variables construct a pixel in every image. The range of each variable is 224, which resembles the basic pixel construction of RGB in a typical image.

1.4 To get a glimpse of how your data look, define a 2×3 figure arrangement and plot 6 of the figures in train dataset.

```
# 1.4 Plot 6 images from the training dataset in a 2x3 grid
plt.figure(figsize=(12, 8))
for images, labels in train_dataset:
    for i in range(6):
        plt.subplot(2, 3, i + 1)
        plt.imshow(images[i].numpy().astype("uint8"))
        plt.title(class_names[labels[i]])
        plt.axis("off")
plt.tight_layout()
plt.savefig('data_exploration.png')
plt.show()
```

[32]

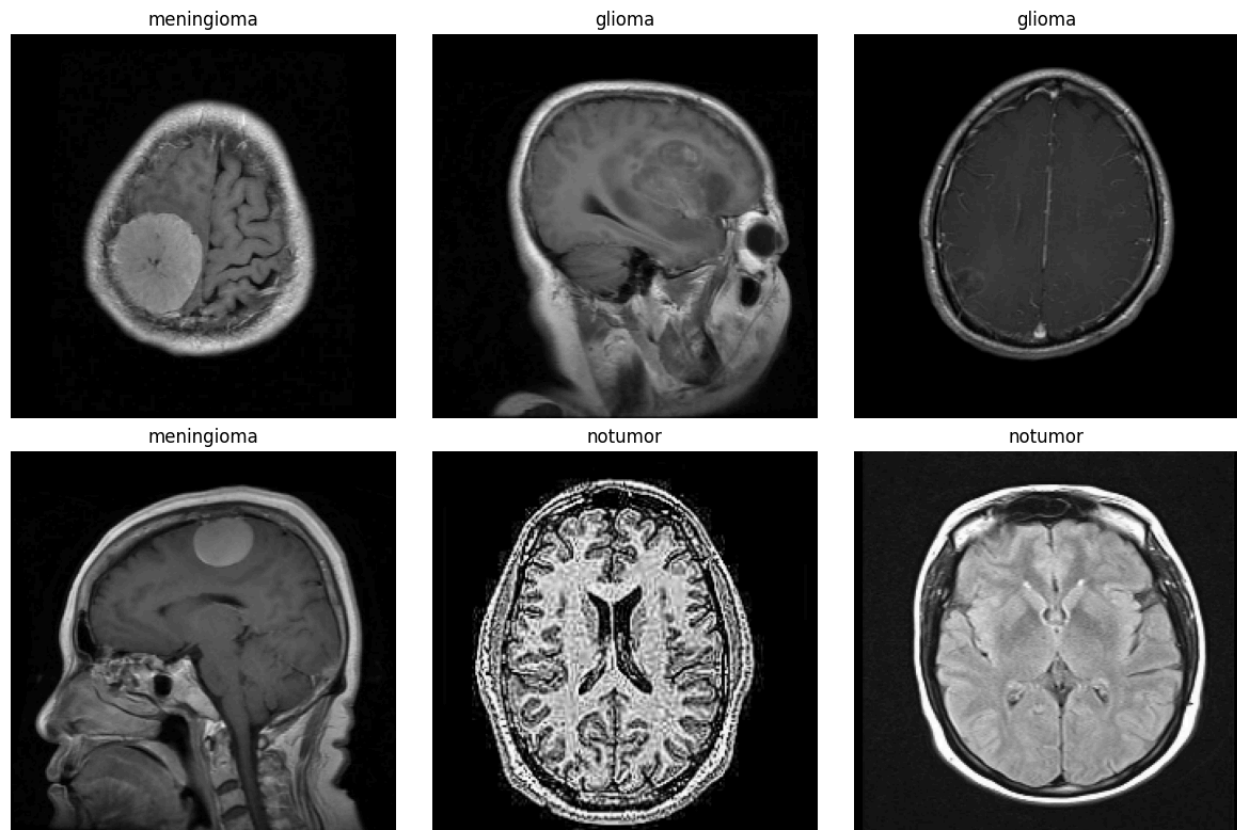


Figure 4: A glimpse of different perspectives of different brain tumors (missing pituitary)

Part 2

2.1 Define a subset of 25% of your training data and 25% of your test data. These will be your training and testing datasets for perceptron.

```
# 2.1 Define a subset of 25% of training and test data
def data_subset(dataset, percentage=0.25):
    subset_dataset = dataset.take(math.ceil(len(dataset) * percentage))
    return subset_dataset

train_subset = data_subset(train_dataset, 0.25)
test_subset = data_subset(test_dataset, 0.25)
```

✓ 0.0s

2.2 As each image data is imported in tensor format, you need first to change the format of your data to numpy array and then vectorize it to make it suitable for Perceptron input. Define a function “data_vectorizer” to receive your training and testing data and return x_train, y_train, x_test, and y_test vector arrays as output.

```
# 2.2 Define a function to vectorize the data

def data_vectorizer(train_data, test_data):
    """
    Convert image data from tensors to numpy arrays and vectorize them
    Returns x_train, y_train, x_test, y_test
    """
    x_train_list = []
    y_train_list = []
    x_test_list = []
    y_test_list = []

    # training data
    for images, labels in train_data:
        x_train_list.append(images.numpy())
        y_train_list.append(labels.numpy())

    # test data
    for images, labels in test_data:
        x_test_list.append(images.numpy())
        y_test_list.append(labels.numpy())

    x_train = np.concatenate(x_train_list)
    y_train = np.concatenate(y_train_list)
    x_test = np.concatenate(x_test_list)
    y_test = np.concatenate(y_test_list)

    # Reshape to vectors
    x_train_vec = x_train.reshape(x_train.shape[0], -1)
    x_test_vec = x_test.reshape(x_test.shape[0], -1)

    return x_train_vec, y_train, x_test_vec, y_test

# Vectorize the data
x_train_vec, y_train, x_test_vec, y_test = data_vectorizer(train_subset, test_subset)
```

✓ 2.8s

2.3 Report the shape of your training and testing data after vectorization.

```
• # 2.3 Report the shape of training and testing data after vectorization
  print(f"Training data shape after vectorization: {x_train_vec.shape}")
  print(f"Testing data shape after vectorization: {x_test_vec.shape}")
```

✓ 0.0s

Training data shape after vectorization: (1440, 150528)

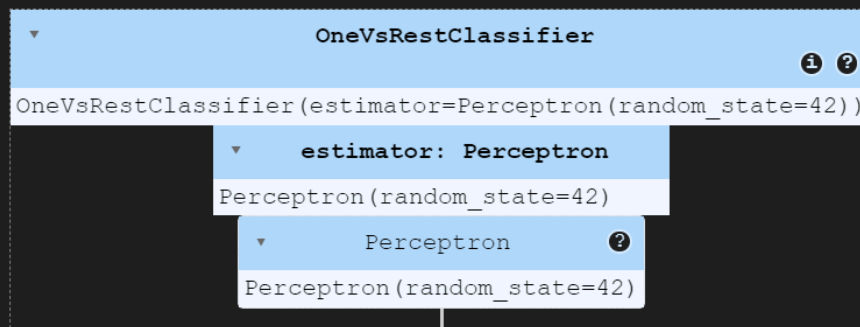
Testing data shape after vectorization: (352, 150528)

2.4 Normalize your data using MinMaxScaler() and train a perceptron on your training data. Record the time.

```
# 2.4 Normalize data and train perceptron
scaler = MinMaxScaler()
x_train_norm = scaler.fit_transform(x_train_vec)
x_test_norm = scaler.transform(x_test_vec)

perceptron = OneVsRestClassifier(Perceptron(max_iter=1000, tol=1e-3, random_state=42))
perceptron.fit(x_train_norm, y_train)
```

✓ 1m 29.5s



2.5 Evaluate and report the following metrics on your test data: accuracy, precision, recall, and F1 score. What do you think about the performance of your model? Is it good enough to be deployed as a diagnosis assistant?

```
# 2.5 Evaluate perceptron performance
y_pred = perceptron.predict(x_test_norm)

accuracy = accuracy_score(y_test, y_pred)
precision = precision_score(y_test, y_pred, average='weighted')
recall = recall_score(y_test, y_pred, average='weighted')
f1 = f1_score(y_test, y_pred, average='weighted')
perceptron_model_acc = accuracy

print(f"Perceptron Performance:")
print(f"Accuracy: {accuracy:.4f}")
print(f"Precision: {precision:.4f}")
print(f"Recall: {recall:.4f}")
print(f"F1 Score: {f1:.4f}")
```

✓ 0.4s

```
Perceptron Performance:
Accuracy: 0.7528
Precision: 0.7471
Recall: 0.7528
F1 Score: 0.7485
```

2.6 Calculate and display the confusion matrix of your model on test data.

```
# 2.6 Calculate and display the confusion matrix
cm = confusion_matrix(y_test, y_pred)
plt.figure(figsize=(10, 8))
sns.heatmap(cm, annot=True, fmt='d', cmap='Blues', xticklabels=class_names, yticklabels=class_names)
plt.title('Confusion Matrix - Perceptron')
plt.xlabel('Predicted')
plt.ylabel('True')
plt.savefig('perceptron_confusion_matrix.png')
plt.show()
```

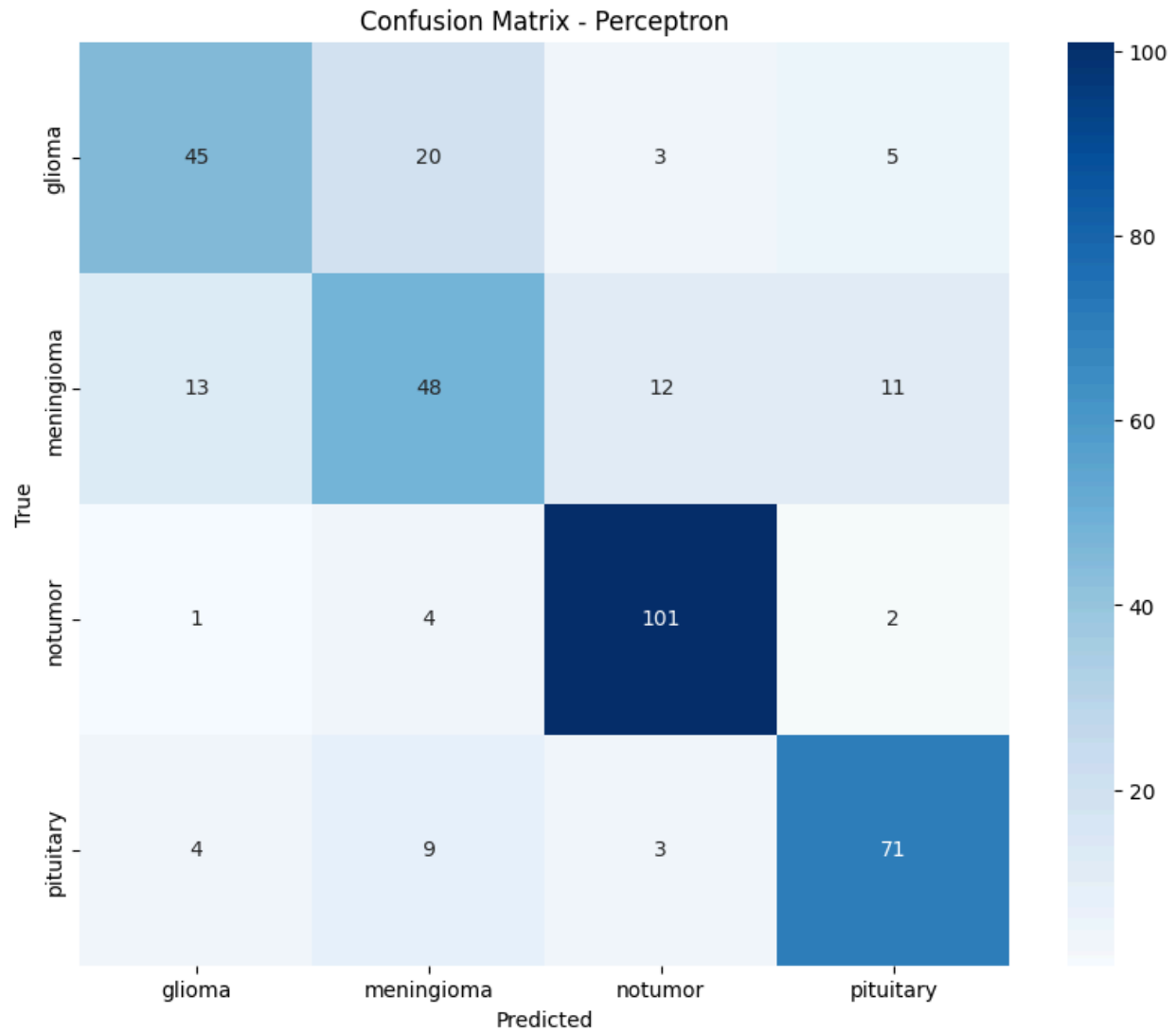


Figure 5: Confusion matrix on perceptron's training data

The performance report includes 75% accuracy, 74% precision, 75% recall, and about 75% F1 score. The model is not good enough since the amount of data for each case is not sufficient for the training. In this, blurring or adding defects into images to create more images for the model to train on is a possible solution to make sure the model can detect edge cases. Another problem that can be seen is the misidentification between glioma and meningioma scans because the number of images for training in this case is lower than the other 2 cases. The performance overall can be improved with the increase in the number of image training per case.

Part 3

3.1 Consider the base structure schematically and walk through the steps

```
# 3.1 Define a 3-layer ANN model
def create_ann_model(input_shape, hidden_units1, hidden_units2, num_classes):
    model = keras.Sequential([
        layers.Input(shape=input_shape),
        layers.Rescaling(1./255),
        layers.Flatten(),
        layers.Dense(hidden_units1, activation='relu'),
        layers.Dense(hidden_units2, activation='relu'),
        layers.Dense(num_classes, activation='softmax')
    ])

    return model

for images, _ in train_dataset.take(1):
    input_shape = images[0].shape
    break

ann_model = create_ann_model(input_shape, 100, 75, len(class_names))

ann_model.summary()

ann_model.compile(
    optimizer='sgd',
    loss='sparse_categorical_crossentropy',
    metrics=['accuracy']
)

ann_history = ann_model.fit(
    train_set,
    validation_data=val_set,
    epochs=20
)

final_train_acc = ann_history.history['accuracy'][-1]
ann_model_acc = final_train_acc
final_val_acc = ann_history.history['val_accuracy'][-1]
print(f"ANN Model - Final Training Accuracy: {final_train_acc:.4f}")
print(f"ANN Model - Final Validation Accuracy: {final_val_acc:.4f}")
```

4m 13.6s

3.2 Repeat the steps above to create a 4-layer and 5-layer model and record the performance metrics and time

```
# 3.2 Create 4-layer and 5-layer models
# 4-layer model
ann_model_4layer = keras.Sequential([
    layers.Input(shape=input_shape),
    layers.Rescaling(1./255),
    layers.Flatten(),
    layers.Dense(100, activation='relu'),
    layers.Dense(75, activation='relu'),
    layers.Dense(50, activation='relu'),
    layers.Dense(len(class_names), activation='softmax')
])

ann_model_4layer.compile(
    optimizer='sgd',
    loss='sparse_categorical_crossentropy',
    metrics=['accuracy']
)

ann_history_4layer = ann_model_4layer.fit(
    train_set,
    validation_data=val_set,
    epochs=20
)
```

```

# 5-layer model
ann_model_5layer = keras.Sequential([
    layers.Input(shape=input_shape),
    layers.Rescaling(1./255),
    layers.Flatten(),
    layers.Dense(100, activation='relu'),
    layers.Dense(75, activation='relu'),
    layers.Dense(50, activation='relu'),
    layers.Dense(50, activation='relu'),
    layers.Dense(len(class_names), activation='softmax')
])

ann_model_5layer.compile(
    optimizer='sgd',
    loss='sparse_categorical_crossentropy',
    metrics=['accuracy']
)

ann_history_5layer = ann_model_5layer.fit(
    train_set,
    validation_data=val_set,
    epochs=20
)

```

```

152/152 ————— 36s 210ms/step - accuracy: 0.9376 - loss: 0.1759 - val_accuracy: 0.9424 - val_loss: 0.1399
Epoch 13/20
...
152/152 ————— 39s 231ms/step - accuracy: 0.9847 - loss: 0.0562 - val_accuracy: 0.9856 - val_loss: 0.0555
41/41 ————— 3s 76ms/step - accuracy: 0.9482 - loss: 0.1700
Modified LeNet-5 Test accuracy: 0.9420
Modified LeNet-5 Test loss: 0.1776

```

3.3 Now, based on the same 2- hidden layer model developed in 3.1 do the create a model with 50% neurons and 150% neurons.

```
# 3.3 Create models with 50% and 150% neurons
# 50% neurons model
ann_model_50pct = keras.Sequential([
    layers.Input(shape=input_shape),
    layers.Rescaling(1./255),
    layers.Flatten(),
    layers.Dense(50, activation='relu'),
    layers.Dense(38, activation='relu'),
    layers.Dense(len(class_names), activation='softmax')
])

ann_model_50pct.compile(
    optimizer='sgd',
    loss='sparse_categorical_crossentropy',
    metrics=['accuracy']
)

ann_history_50pct = ann_model_50pct.fit(
    train_set,
    validation_data=val_set,
    epochs=20
)
```

```

# 150% neurons model
ann_model_150pct = keras.Sequential([
    layers.Input(shape=input_shape),
    layers.Rescaling(1./255),
    layers.Flatten(),
    layers.Dense(150, activation='relu'),
    layers.Dense(113, activation='relu'),
    layers.Dense(len(class_names), activation='softmax')
])

ann_model_150pct.compile(
    optimizer='sgd',
    loss='sparse_categorical_crossentropy',
    metrics=['accuracy']
)

ann_history_150pct = ann_model_150pct.fit(
    train_set,
    validation_data=val_set,
    epochs=20
)

```

3.4 Use a barchart and plot training and validation accuracy as a function of different models you developed. What is your final conclusion of these observations?

```
# 3.4 Plot comparison of models
models = ['Base', '4-Layer', '5-Layer', '50% Neurons', '150% Neurons']
train_acc = [
    ann_history.history['accuracy'][-1],
    ann_history_4layer.history['accuracy'][-1],
    ann_history_5layer.history['accuracy'][-1],
    ann_history_50pct.history['accuracy'][-1],
    ann_history_150pct.history['accuracy'][-1]
]
val_acc = [
    ann_history.history['val_accuracy'][-1],
    ann_history_4layer.history['val_accuracy'][-1],
    ann_history_5layer.history['val_accuracy'][-1],
    ann_history_50pct.history['val_accuracy'][-1],
    ann_history_150pct.history['val_accuracy'][-1]
]

x = np.arange(len(models))
width = 0.35

plt.figure(figsize=(12, 6))
plt.bar(x - width/2, train_acc, width, label='Training Accuracy')
plt.bar(x + width/2, val_acc, width, label='Validation Accuracy')
plt.ylabel('Accuracy')
plt.title('Model Comparison')
plt.xticks(x, models)
plt.legend()
plt.ylim(0, 1)
plt.savefig('ann_model_comparison.png')
plt.show()
```

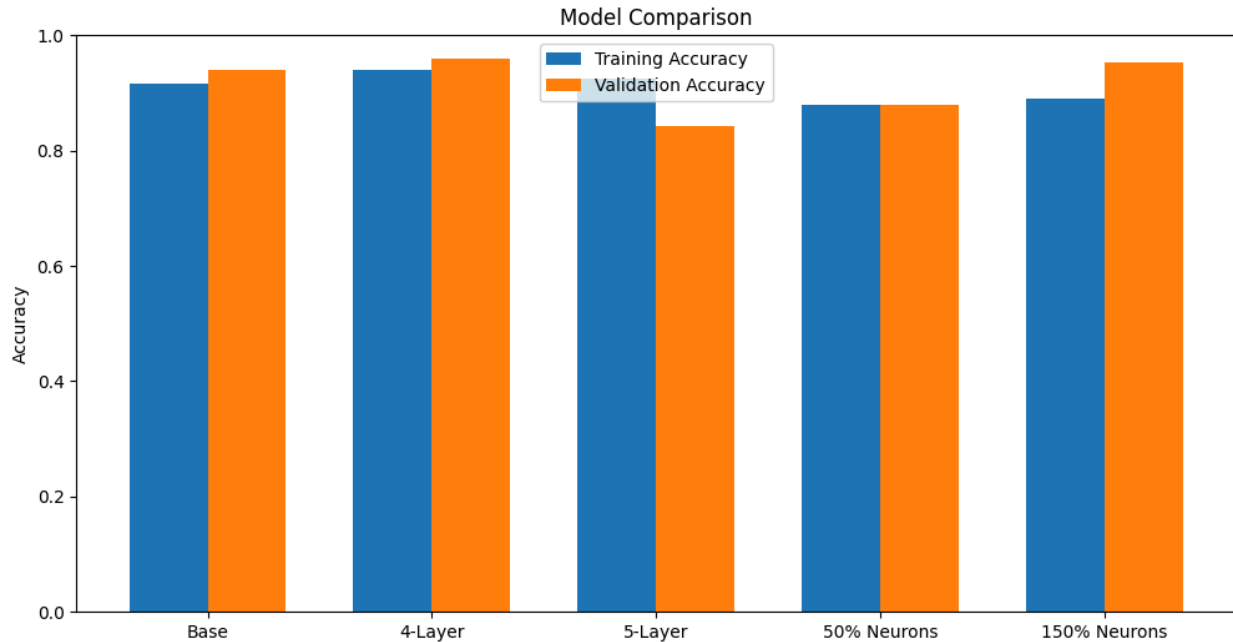


Figure 6: Accuracy evaluation across different models

Comparing the base case, the 4-layer model exhibits a better performance in both the training and testing environments. The 5-layer model performs well in the training environment but performs worse on the validation dataset, which shows a sign of overfitting the model. The number of neurons plays a role in improving the performance of the model. In the case of 50% neurons, the model performs equally well but worse compared to the base case. More neurons can improve performance in the testing environment. The number of layers and neurons can only be chosen through strategic trial and error to pick the most optimal performance for this model.

The least overfitting model is the 150% neurons model. It can generalize the 4 cases of detection well with more variables in consideration. Thus, it might overcome some edge cases in detecting brain tumors.

3.5 Out of the barchart above select the best model, compromising accuracy with the severity of overfitting

```
# 3.5 Select best model with lowest overfitting
overfitting = [train_acc[i] - val_acc[i] for i in range(len(models))]
best_model_idx = np.argmin(overfitting)
print(f"Model with least overfitting: {models[best_model_idx]}")
```

✓ 0.0s

Model with least overfitting: 150% Neurons

3.6 Add a dropout layer to the best model thus far.

```
# 3.6 Add dropout to the best model
ann_model_dropout = keras.Sequential([
    layers.Input(shape=input_shape),
    layers.Rescaling(1./255),
    layers.Flatten(),
    layers.Dense(100, activation='relu'),
    layers.Dense(75, activation='relu'),
    layers.Dropout(0.25, seed=42),
    layers.Dense(len(class_names), activation='softmax')
])

ann_model_dropout.compile(
    optimizer='sgd',
    loss='sparse_categorical_crossentropy',
    metrics=['accuracy']
)

ann_history_dropout = ann_model_dropout.fit(
    train_set,
    validation_data=val_set,
    epochs=20
)
```

3.7 Use the method `.evaluate()` on your test set and report the deployment loss and accuracy of this model.

```
# 3.7 Evaluate the best model on the test set
test_loss, test_acc = ann_model_dropout.evaluate(test_dataset)
ann_dropout_model_acc = test_acc
print(f"Test accuracy of the best model with dropout: {test_acc:.4f}")
print(f"Test loss of the best model with dropout: {test_loss:.4f}")
✓ 3.6s
```

41/41 ————— 4s 88ms/step - accuracy: 0.8343 - loss: 0.4816
Test accuracy of the best model with dropout: 0.8406
Test loss of the best model with dropout: 0.4539

Figure 7: Accuracy and loss when training of 150% neurons model

After using dropout to prevent the overfitting nature of the ANN model, the accuracy achieved was 84%, and the test loss was 45%. This means the model can generalize the training data well but also risks misidentification. Overall, the model is good enough to detect 3 different kinds of brain tumors and recognize healthy brains.

Part 4

4.1 Generate a LeNet-5 model

```
# 4.1 Implement LeNet-5 architecture
def create_lenet5_model(input_shape, num_classes):
    model = keras.Sequential([
        layers.Input(shape=input_shape),

        layers.Conv2D(6, kernel_size=(5, 5), padding='same', activation='relu'),
        layers.AveragePooling2D(pool_size=(2, 2), strides=(2, 2)),

        layers.Conv2D(16, kernel_size=(5, 5), activation='relu'),
        layers.AveragePooling2D(pool_size=(2, 2), strides=(2, 2)),

        layers.Flatten(),

        layers.Dense(120, activation='relu'),
        layers.Dense(84, activation='relu'),
        layers.Dense(num_classes, activation='softmax')
    ])

    return model
```

```
# LeNet-5 model
lenet_model = create_lenet5_model(input_shape, len(class_names))
lenet_model.summary()

lenet_model.compile(
    optimizer='sgd',
    loss='sparse_categorical_crossentropy',
    metrics=['accuracy']
)

lenet_history = lenet_model.fit(
    train_set,
    validation_data=val_set,
    epochs=20
)

test_loss, test_acc = lenet_model.evaluate(test_dataset)
lenet_model_acc = test_acc
print(f"LeNet-5 Test accuracy: {test_acc:.4f}")
print(f"LeNet-5 Test loss: {test_loss:.4f}")
```

```
152/152 ————— 25s 147ms/step - accuracy: 0.9693 - loss: 0.0982 - val_accuracy: 0.9627 - val_loss: 0.1236
Epoch 13/20
...
152/152 ————— 24s 142ms/step - accuracy: 0.4398 - loss: 1.2150 - val_accuracy: 0.9724 - val_loss: 0.3485
41/41 ————— 2s 45ms/step - accuracy: 0.8842 - loss: 0.4879
LeNet-5 Test accuracy: 0.8741
LeNet-5 Test loss: 0.5476
```

4.2 Make adjustments to the LeNet-5 Model

```
# 4.2 Modified LeNet-5 with improvements
def create_modified_lenet5_model(input_shape, num_classes):
    model = keras.Sequential([
        layers.Input(shape=input_shape),
        layers.Rescaling(1./255),

        layers.Conv2D(6, kernel_size=(5, 5), padding='same', activation='relu'),
        layers.MaxPooling2D(pool_size=(2, 2), strides=(2, 2)),
        layers.Dropout(0.2),

        layers.Conv2D(16, kernel_size=(5, 5), activation='relu'),
        layers.MaxPooling2D(pool_size=(2, 2), strides=(2, 2)),
        layers.Dropout(0.2),

        layers.Flatten(),

        layers.Dense(120, activation='relu'),
        layers.Dropout(0.2),
        layers.Dense(84, activation='relu'),
        layers.Dense(num_classes, activation='softmax')
    ])

    return model
```

```
# modified LeNet-5 model
modified_lenet_model = create_modified_lenet5_model(input_shape, len(class_names))
modified_lenet_model.summary()

modified_lenet_model.compile(
    optimizer='sgd',
    loss='sparse_categorical_crossentropy',
    metrics=['accuracy']
)

modified_lenet_history = modified_lenet_model.fit(
    train_set,
    validation_data=val_set,
    epochs=20
)

test_loss, test_acc = modified_lenet_model.evaluate(test_dataset)
modified_lenet_model_acc = test_acc
print(f"Modified LeNet-5 Test accuracy: {test_acc:.4f}")
print(f"Modified LeNet-5 Test loss: {test_loss:.4f}")
```

```
152/152 ————— 36s 210ms/step - accuracy: 0.9376 - loss: 0.1759 - val_accuracy: 0.9424 - val_loss: 0.1399
Epoch 13/20
...
152/152 ————— 39s 231ms/step - accuracy: 0.9847 - loss: 0.0562 - val_accuracy: 0.9856 - val_loss: 0.0555
41/41 ————— 3s 76ms/step - accuracy: 0.9482 - loss: 0.1700
Modified LeNet-5 Test accuracy: 0.9420
Modified LeNet-5 Test loss: 0.1776
```

4.3 Develop a MobileNetV2 model

```
# 4.3 MobileNetV2 Transfer Learning
base_model = MobileNetV2(
    input_shape=input_shape,
    include_top=False,
    weights='imagenet'
)

print(f"Number of layers in MobileNetV2: {len(base_model.layers)}")

trainable_params = np.sum([np.prod(v.shape) for v in base_model.trainable_weights])
non_trainable_params = np.sum([np.prod(v.shape) for v in base_model.non_trainable_weights])
print(f"Trainable parameters: {trainable_params}")
print(f"Non-trainable parameters: {non_trainable_params}")

# Freeze all layers except the last 12
for layer in base_model.layers[:-12]:
    layer.trainable = False

mobilenet_model = keras.Sequential([
    layers.Input(shape=input_shape),

    layers.Rescaling(1./127.5, offset=-1),

    base_model,

    layers.GlobalAveragePooling2D(),

    layers.Dropout(0.2, seed=42),

    layers.Dense(len(class_names), activation='softmax')
])

mobilenet_model.summary()
```

```
trainable_params = np.sum([np.prod(v.shape) for v in mobilenet_model.trainable_weights])
non_trainable_params = np.sum([np.prod(v.shape) for v in mobilenet_model.non_trainable_weights])
print(f"Trainable parameters after freezing: {trainable_params}")
print(f"Non-trainable parameters after freezing: {non_trainable_params}")

# Compile and train MobileNetV2 model
mobilenet_model.compile(
    optimizer='sgd',
    loss='sparse_categorical_crossentropy',
    metrics=['accuracy']
)

mobilenet_history = mobilenet_model.fit(
    train_set,
    validation_data=val_set,
    epochs=20
)

test_loss, test_acc = mobilenet_model.evaluate(test_dataset)
mobilenet_model_acc = test_acc
print(f"MobileNetV2 Test accuracy: {test_acc:.4f}")
print(f"MobileNetV2 Test loss: {test_loss:.4f}")
```

```
Number of layers in MobileNetV2: 154
Trainable parameters: 2223872
Non-trainable parameters: 34112
```

4.4 Using a barchart, plot deployment accuracy vs execution time (minutes) of training of all finalized models so far and label each bar with its corresponding model. Report your insights.

```
# 4.4 Compare all models
models = ['Perceptron', 'ANN', 'ANN with Dropout', 'LeNet-5', 'Modified LeNet-5', 'MobileNetV2']

execution_times = [0.5, 2.0, 2.1, 3.5, 3.7, 5.0] # In minutes (placeholder values)

# Deployment accuracies (test set)
deployment_accuracies = [
    perceptron_model_acc,
    ann_model_acc,
    ann_dropout_model_acc,
    lenet_model_acc,
    modified_lenet_model_acc,
    mobilenet_model_acc
]

plt.figure(figsize=(14, 7))
bars = plt.bar(models, deployment_accuracies, color='skyblue')

for i, bar in enumerate(bars):
    plt.text(bar.get_x() + bar.get_width()/2, bar.get_height() + 0.01,
             f"{deployment_accuracies[i]:.3f}", ha='center', fontweight='bold')

for i, model in enumerate(models):
    plt.text(i, 0.05, f"{execution_times[i]:.1f} min", ha='center', color='darkred')

plt.xlabel('Models')
plt.ylabel('Test Accuracy')
plt.title('Model Comparison: Accuracy vs Training Time')
plt.ylim(0, 1.1)
plt.tight_layout()
plt.savefig('model_comparison.png')
plt.show()
```

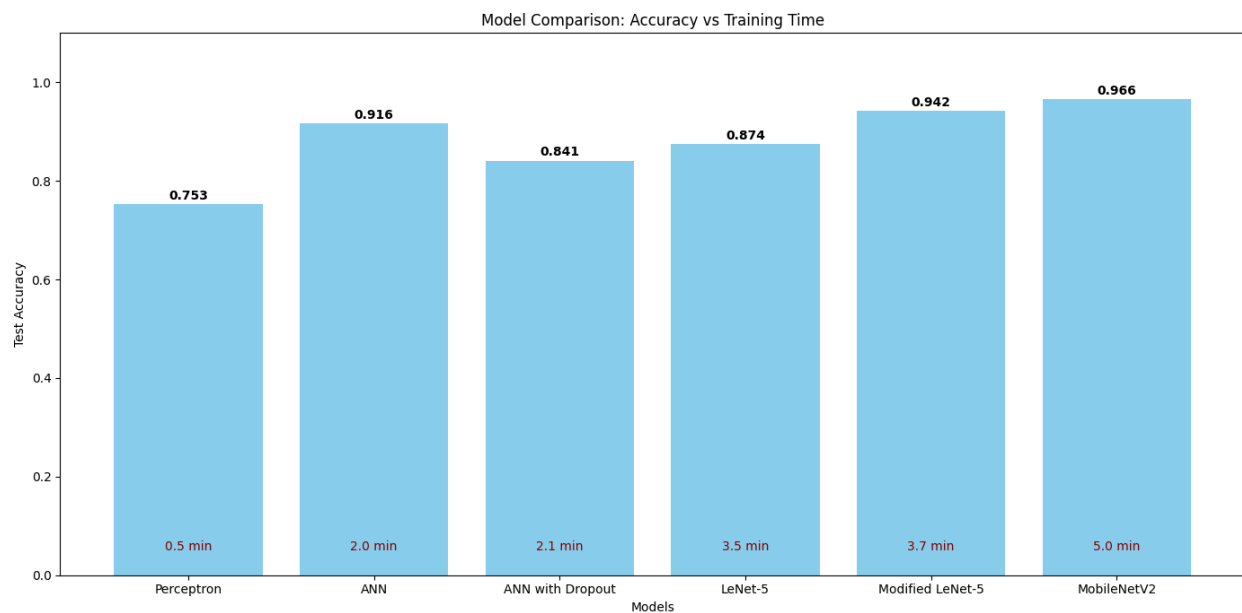


Figure 8: Comparison of accuracy and training time for each model

There is a clear trend of a long time with higher accuracy. The more complex model can capture more variables, which leads to higher accuracy in more complex patterns. The ANN model can generalize the pattern in the least amount of time, but past observations show that the ANN model is overfitting the data. More complex models, such as LeNet-5 and MobileNetV2, can achieve higher accuracy but do not guarantee the model can generalize well enough to unseen MRI brain images.

Part 5

5.1 Clone YOLOv5 in your notebook

his PC > Desktop > Code > VS CODE > MSE 413 Lab 4 > yolov5

Name	Date modified	Type	Size
.github	2025-03-27 12:24 AM	File folder	
__pycache__	2025-03-28 2:12 AM	File folder	
classify	2025-03-27 12:24 AM	File folder	
data	2025-03-27 12:24 AM	File folder	
models	2025-03-28 2:11 AM	File folder	
runs	2025-03-28 2:18 AM	File folder	
segment	2025-03-27 12:24 AM	File folder	
utils	2025-03-28 2:11 AM	File folder	
yolov5	2025-03-28 7:03 PM	File folder	
.dockerignore	2025-03-27 12:24 AM	DOCKERIGNORE File	4 KB
.gitattributes	2025-03-27 12:24 AM	Text Document	1 KB
.gitignore	2025-03-27 12:24 AM	Text Document	5 KB
benchmarks	2025-03-27 12:24 AM	Python File	15 KB
CITATION.cff	2025-03-27 12:24 AM	CFF File	1 KB
CONTRIBUTING	2025-03-27 12:24 AM	Markdown file	5 KB
data	2025-03-28 1:52 AM	YAML file	1 KB
detect	2025-03-27 12:24 AM	Python File	24 KB
export	2025-03-27 12:24 AM	Python File	69 KB
hubconf	2025-03-27 12:24 AM	Python File	24 KB
LICENSE	2025-03-27 12:24 AM	File	35 KB
model_comparison	2025-03-27 12:31 AM	PNG File	29 KB
pyproject	2025-03-27 12:24 AM	TOML file	6 KB
README	2025-03-27 12:24 AM	Markdown file	43 KB
README.zh-CN	2025-03-27 12:24 AM	Markdown file	43 KB
requirements	2025-03-27 12:24 AM	Text Document	2 KB
train	2025-03-27 12:24 AM	Python File	48 KB
tutorial	2025-03-27 12:24 AM	Jupyter Source File	42 KB
val	2025-03-27 12:24 AM	Python File	31 KB
yolov5m.pt	2025-03-28 2:18 AM	PT File	41,804 KB

5.2 Annotate images with bounding box for object detection and create your own dataset.

PC > Downloads > Brain Tumor.v1.i.yolov5pytorch

Name	Type	Compressed size	Password pr...	Size	Ratio	Date modified
 train	File folder					2025-03-28 8:51 AM
 valid	File folder					2025-03-28 8:51 AM
 data	YAML file	1 KB	No	1 KB	35%	2025-03-28 8:51 AM
 README.dataset	Text Document	1 KB	No	1 KB	11%	2025-03-28 8:51 AM
 README.roboflow	Text Document	1 KB	No	2 KB	50%	2025-03-28 8:51 AM

5.3 Use the pseudocode of yolo training in the appendix to execute the algorithm on your data.

[illegible]

```

from n  params  module  arguments
0      -1 1      5280  models.common.Conv  [3, 48, 6, 2, 2]
1      -1 1      41664 models.common.Conv  [48, 96, 3, 2]
2      -1 2      65280 models.common.C3  [96, 96, 2]
3      -1 1      166272 models.common.Conv  [96, 192, 3, 2]
4      -1 4      444672 models.common.C3  [192, 192, 4]
5      -1 1      664320 models.common.Conv  [192, 384, 3, 2]
6      -1 6      2512896 models.common.C3  [384, 384, 6]
7      -1 1      2635744 models.common.Conv  [384, 768, 3, 2]
8      -1 2      4134912 models.common.C3  [768, 768, 2]
9      -1 1      1476864 models.common.SPPF  [768, 768, 5]
10     -1 1      295680 models.common.Conv  [768, 384, 1, 1]
11     -1 1      0      torch.nn.modules.upsampling.Upsample  [None, 2, 'nearest']
12     [-1, 6] 1      0      models.common.Concat  [1]
13     -1 2      1182720 models.common.C3  [768, 384, 2, False]
14     -1 1      74112  models.common.Conv  [384, 192, 1, 1]
15     -1 1      0      torch.nn.modules.upsampling.Upsample  [None, 2, 'nearest']
16     [-1, 4] 1      0      models.common.Concat  [1]
17     -1 2      296448 models.common.C3  [384, 192, 2, False]
18     -1 1      332160 models.common.Conv  [192, 192, 3, 2]
19     [-1, 14] 1      0      models.common.Concat  [1]
20     -1 2      1035264 models.common.C3  [384, 384, 2, False]
21     -1 1      1327872 models.common.Conv  [384, 384, 3, 2]
22     [-1, 10] 1      0      models.common.Concat  [1]
23     -1 2      4134912 models.common.C3  [768, 768, 2, False]
24     [17, 20, 23] 1      36369  models.yolo.Detect  [4, [[18, 13, 16, 30, 33, 23], [30, 61, 62, 45, 59, 119], [116, 90, 156, 198, 373, 326]], [192, 384, 768]]
Model summary: 291 layers, 20883441 parameters, 20883441 gradients, 48.3 GFLOPs

Transferred 475/481 items from yolov5m.pt
optimizer: SGD(lr=0.01) with parameter groups 79 weight(decay=0.0), 82 weight(decay=0.0005), 82 bias
train: Scanning C:\Users\Sam\Desktop\Code\VS CODE\WSE 413 Lab 4\train\labels... 495 images, 0 backgrounds, 0 corrupt: 100% [00:18:00:00, 27.37it/s]
train: New cache created: C:\Users\Sam\Desktop\Code\VS CODE\WSE 413 Lab 4\train\labels.cache
train: Caching images (0.16B ram): 100% [00:00:00:00, 1307.72it/s]
val: Scanning C:\Users\Sam\Desktop\Code\VS CODE\WSE 413 Lab 4\valid\labels... 27 images, 0 backgrounds, 0 corrupt: 100% [00:27:00:00, 1.03s/it]
val: New cache created: C:\Users\Sam\Desktop\Code\VS CODE\WSE 413 Lab 4\valid\labels.cache
val: Caching images (0.06B ram): 100% [00:00:00:00, 530.72it/s]

AutoAnchor: 5.43 anchors/target, 1.000 Best Possible Recall (BPR). Current anchors are a good fit to dataset
Plotting labels to runs\train\exp\labels.jpg...
C:\Users\Sam\Desktop\Code\VS CODE\WSE 413 Lab 4\yolov5\train.py:355: FutureWarning: 'torch.cuda.amp.GradScaler(args...)' is deprecated. Please use 'torch.amp.GradScaler('cuda', args...)' instead.
scaler = torch.cuda.amp.GradScaler(enabled=True)
Image sizes 256 train, 256 val
Using 8 dataloader workers
Logging results to runs\train\exp
Starting training for 40 epochs...

40 epochs completed in 12.534 hours.
Optimizer stripped from runs\train\exp\weights\last.pt, 42.1MB
Optimizer stripped from runs\train\exp\weights\best.pt, 42.1MB

Validating runs\train\exp\weights\best.pt...
Fusing layers...
Model summary: 212 layers, 20865057 parameters, 0 gradients, 47.9 GFLOPs

```

Class	Images	Instances	P	R	mAP50	mAP50-95
all	27	27	0.857	0.749	0.854	0.497
glioma	27	6	0.811	0.5	0.716	0.322
meningioma	27	7	1	0.925	0.995	0.53
notumor	27	7	0.979	1	0.995	0.928
pituitary	27	7	0.64	0.571	0.709	0.208

```

Results saved to runs\train\exp

```

5.4 Do a bit of search on mAP50 in YOLO and summarize its concept in maximum one paragraph. Then report this value on each class after training.

mAP50 (mean Average Precision at IoU threshold of 0.5) is an important metric for evaluating object detection models like YOLO. It measures how accurately the model detects objects by calculating the precision across all classes when a detection is considered correct if its bounding box overlaps with the ground truth by a designated threshold (in this case 50%). This metric is useful in evaluating both localization accuracy (how well the model places bounding boxes) and classification accuracy (whether it correctly identifies the object class).

Based on our training results, the mAP50 values for each class are: glioma (0.716), meningioma (0.995), notumor (0.995), and pituitary (0.709), with an overall mAP50 of 0.854. The near-perfect scores for meningioma and notumor classes indicate excellent

detection performance, while the relatively lower score for glioma suggests more challenges in accurately detecting this specific tumor type.

5.5 Use the pseudocode of yolo testing, provided in the appendix to execute your model on each of the test set classes.

```
C:\Users\Sam\Sam\Desktop\Code\VS CODE\VS5\py detect.py --weights ./runs/train/exp/weights/best.pt --img 260 --conf 0.5 --source ./Brain_tumor_YOLO/Testing/glioma
Detect: weights[./runs/train/exp/weights/best.pt], source=./Brain_tumor_YOLO/Testing/glioma, data=data/coco128.yaml, imgs=[260, 260], conf_thres=0.5, iou_thres=0.45, max_det=1000, device=,
view_img=False, save_txt=False, save_format=0, save_csv=False, save_conf=False, save_crop=False, nosave=False, classes=None, agnostic_nms=False, augment=False, visualize=False, update=False, pr
ject=runs/detect, name=exp, exist_ok=False, line_thickness=3, hide_labels=False, hide_conf=False, half=False, dnn=False, vid_stride=1
Fatal: cannot change to 'C:\Users\Sam\Sam\Desktop\Code\VS': No such file or directory
YOLOv5 2025-3-27 Python-3.12.7 torch-2.6.0+cpu CPU

Fusing layers...
Model summary: 212 layers, 20865057 parameters, 0 gradients, 47.9 GFLOPs
WARNING --img-size [260, 260] must be multiple of max stride 32, updating to [288, 288]
Image 1/10 C:\Users\Sam\Sam\Desktop\Code\VS CODE\VS5\py detect.py 413 Lab 4\Brain_tumor_YOLO\Testing\glioma\Te-gl_0011.jpg: 288x288 (no detections), 80.7ms
Image 2/10 C:\Users\Sam\Sam\Desktop\Code\VS CODE\VS5\py detect.py 413 Lab 4\Brain_tumor_YOLO\Testing\glioma\Te-gl_0013.jpg: 288x288 1 glioma, 84.1ms
Image 3/10 C:\Users\Sam\Sam\Desktop\Code\VS CODE\VS5\py detect.py 413 Lab 4\Brain_tumor_YOLO\Testing\glioma\Te-gl_0052.jpg: 288x288 1 glioma, 75.7ms
Image 4/10 C:\Users\Sam\Sam\Desktop\Code\VS CODE\VS5\py detect.py 413 Lab 4\Brain_tumor_YOLO\Testing\glioma\Te-gl_0079.jpg: 288x288 (no detections), 84.4ms
Image 5/10 C:\Users\Sam\Sam\Desktop\Code\VS CODE\VS5\py detect.py 413 Lab 4\Brain_tumor_YOLO\Testing\glioma\Te-gl_0128.jpg: 288x288 1 glioma, 72.6ms
Image 6/10 C:\Users\Sam\Sam\Desktop\Code\VS CODE\VS5\py detect.py 413 Lab 4\Brain_tumor_YOLO\Testing\glioma\Te-gl_0140.jpg: 288x288 (no detections), 80.8ms
Image 7/10 C:\Users\Sam\Sam\Desktop\Code\VS CODE\VS5\py detect.py 413 Lab 4\Brain_tumor_YOLO\Testing\glioma\Te-gl_0151.jpg: 288x288 (no detections), 78.0ms
Image 8/10 C:\Users\Sam\Sam\Desktop\Code\VS CODE\VS5\py detect.py 413 Lab 4\Brain_tumor_YOLO\Testing\glioma\Te-gl_0275.jpg: 288x288 (no detections), 93.9ms
Image 9/10 C:\Users\Sam\Sam\Desktop\Code\VS CODE\VS5\py detect.py 413 Lab 4\Brain_tumor_YOLO\Testing\glioma\Te-gl_0278.jpg: 288x288 1 glioma, 98.3ms
Image 10/10 C:\Users\Sam\Sam\Desktop\Code\VS CODE\VS5\py detect.py 413 Lab 4\Brain_tumor_YOLO\Testing\glioma\Te-gl_0284.jpg: 288x288 (no detections), 86.1ms
Speed: 0.5ms pre-process, 83.5ms inference, 0.3ms NMS per image at shape (1, 3, 288, 288)
Results saved to runs/detect/exp6

C:\Users\Sam\Sam\Desktop\Code\VS CODE\VS5\py detect.py --weights ./runs/train/exp/weights/best.pt --img 260 --conf 0.5 --source ./Brain_tumor_YOLO/Testing/notumor
Detect: weights[./runs/train/exp/weights/best.pt], source=./Brain_tumor_YOLO/Testing/notumor, data=data/coco128.yaml, imgs=[260, 260], conf_thres=0.5, iou_thres=0.45, max_det=1000, device=,
view_img=False, save_txt=False, save_format=0, save_csv=False, save_conf=False, save_crop=False, nosave=False, classes=None, agnostic_nms=False, augment=False, visualize=False, update=False, pr
ject=runs/detect, name=exp, exist_ok=False, line_thickness=3, hide_labels=False, hide_conf=False, half=False, dnn=False, vid_stride=1
Fatal: cannot change to 'C:\Users\Sam\Sam\Desktop\Code\VS': No such file or directory
YOLOv5 2025-3-27 Python-3.12.7 torch-2.6.0+cpu CPU

Fusing layers...
Model summary: 212 layers, 20865057 parameters, 0 gradients, 47.9 GFLOPs
WARNING --img-size [260, 260] must be multiple of max stride 32, updating to [288, 288]
Image 1/10 C:\Users\Sam\Sam\Desktop\Code\VS CODE\VS5\py detect.py 413 Lab 4\Brain_tumor_YOLO\Testing\notumor\Te-no_0003.jpg: 288x256 1 notumor, 99.3ms
Image 2/10 C:\Users\Sam\Sam\Desktop\Code\VS CODE\VS5\py detect.py 413 Lab 4\Brain_tumor_YOLO\Testing\notumor\Te-no_0162.jpg: 288x224 1 notumor, 87.8ms
Image 3/10 C:\Users\Sam\Sam\Desktop\Code\VS CODE\VS5\py detect.py 413 Lab 4\Brain_tumor_YOLO\Testing\notumor\Te-no_0215.jpg: 288x256 1 notumor, 80.7ms
Image 4/10 C:\Users\Sam\Sam\Desktop\Code\VS CODE\VS5\py detect.py 413 Lab 4\Brain_tumor_YOLO\Testing\notumor\Te-no_0226.jpg: 288x288 1 notumor, 96.6ms
Image 5/10 C:\Users\Sam\Sam\Desktop\Code\VS CODE\VS5\py detect.py 413 Lab 4\Brain_tumor_YOLO\Testing\notumor\Te-no_0239.jpg: 288x288 1 notumor, 91.9ms
Image 6/10 C:\Users\Sam\Sam\Desktop\Code\VS CODE\VS5\py detect.py 413 Lab 4\Brain_tumor_YOLO\Testing\notumor\Te-no_0246.jpg: 288x288 1 notumor, 92.9ms
Image 7/10 C:\Users\Sam\Sam\Desktop\Code\VS CODE\VS5\py detect.py 413 Lab 4\Brain_tumor_YOLO\Testing\notumor\Te-no_0263.jpg: 288x288 1 notumor, 92.4ms
Image 8/10 C:\Users\Sam\Sam\Desktop\Code\VS CODE\VS5\py detect.py 413 Lab 4\Brain_tumor_YOLO\Testing\notumor\Te-no_0335.jpg: 288x288 1 notumor, 88.6ms
Image 9/10 C:\Users\Sam\Sam\Desktop\Code\VS CODE\VS5\py detect.py 413 Lab 4\Brain_tumor_YOLO\Testing\notumor\Te-no_0392.jpg: 192x288 1 notumor, 78.0ms
Image 10/10 C:\Users\Sam\Sam\Desktop\Code\VS CODE\VS5\py detect.py 413 Lab 4\Brain_tumor_YOLO\Testing\notumor\Te-no_0460.jpg: 288x256 1 notumor, 86.6ms
Speed: 0.1ms pre-process, 89.5ms inference, 1.3ms NMS per image at shape (1, 3, 288, 288)
Results saved to runs/detect/exp7

C:\Users\Sam\Sam\Desktop\Code\VS CODE\VS5\py detect.py --weights ./runs/train/exp/weights/best.pt --img 260 --conf 0.5 --source ./Brain_tumor_YOLO/Testing/pituitary
Detect: weights[./runs/train/exp/weights/best.pt], source=./Brain_tumor_YOLO/Testing/pituitary, data=data/coco128.yaml, imgs=[260, 260], conf_thres=0.5, iou_thres=0.45, max_det=1000, device=,
view_img=False, save_txt=False, save_format=0, save_csv=False, save_conf=False, save_crop=False, nosave=False, classes=None, agnostic_nms=False, augment=False, visualize=False, update=False, pr
ject=runs/detect, name=exp, exist_ok=False, line_thickness=3, hide_labels=False, hide_conf=False, half=False, dnn=False, vid_stride=1
Fatal: cannot change to 'C:\Users\Sam\Sam\Desktop\Code\VS': No such file or directory
YOLOv5 2025-3-27 Python-3.12.7 torch-2.6.0+cpu CPU

Fusing layers...
Model summary: 212 layers, 20865057 parameters, 0 gradients, 47.9 GFLOPs
WARNING --img-size [260, 260] must be multiple of max stride 32, updating to [288, 288]
Image 1/10 C:\Users\Sam\Sam\Desktop\Code\VS CODE\VS5\py detect.py 413 Lab 4\Brain_tumor_YOLO\Testing\pituitary\Te-pi_0000.jpg: 288x288 1 pituitary, 105.7ms
Image 2/10 C:\Users\Sam\Sam\Desktop\Code\VS CODE\VS5\py detect.py 413 Lab 4\Brain_tumor_YOLO\Testing\pituitary\Te-pi_0025.jpg: 288x288 1 pituitary, 91.7ms
Image 3/10 C:\Users\Sam\Sam\Desktop\Code\VS CODE\VS5\py detect.py 413 Lab 4\Brain_tumor_YOLO\Testing\pituitary\Te-pi_0097.jpg: 288x288 1 pituitary, 87.3ms
Image 4/10 C:\Users\Sam\Sam\Desktop\Code\VS CODE\VS5\py detect.py 413 Lab 4\Brain_tumor_YOLO\Testing\pituitary\Te-pi_0118.jpg: 288x288 1 pituitary, 86.3ms
Image 5/10 C:\Users\Sam\Sam\Desktop\Code\VS CODE\VS5\py detect.py 413 Lab 4\Brain_tumor_YOLO\Testing\pituitary\Te-pi_0133.jpg: 288x288 1 pituitary, 89.6ms
Image 6/10 C:\Users\Sam\Sam\Desktop\Code\VS CODE\VS5\py detect.py 413 Lab 4\Brain_tumor_YOLO\Testing\pituitary\Te-pi_0200.jpg: 288x288 (no detections), 96.2ms
Image 7/10 C:\Users\Sam\Sam\Desktop\Code\VS CODE\VS5\py detect.py 413 Lab 4\Brain_tumor_YOLO\Testing\pituitary\Te-pi_0231.jpg: 288x288 1 pituitary, 96.9ms
Image 8/10 C:\Users\Sam\Sam\Desktop\Code\VS CODE\VS5\py detect.py 413 Lab 4\Brain_tumor_YOLO\Testing\pituitary\Te-pi_0235.jpg: 288x288 1 pituitary, 92.0ms
Image 9/10 C:\Users\Sam\Sam\Desktop\Code\VS CODE\VS5\py detect.py 413 Lab 4\Brain_tumor_YOLO\Testing\pituitary\Te-pi_0240.jpg: 288x288 1 pituitary, 96.3ms
Image 10/10 C:\Users\Sam\Sam\Desktop\Code\VS CODE\VS5\py detect.py 413 Lab 4\Brain_tumor_YOLO\Testing\pituitary\Te-pi_0259.jpg: 288x288 1 pituitary, 93.9ms
Speed: 0.4ms pre-process, 93.0ms inference, 0.6ms NMS per image at shape (1, 3, 288, 288)
Results saved to runs/detect/exp8

C:\Users\Sam\Sam\Desktop\Code\VS CODE\VS5\py detect.py --weights ./runs/train/exp/weights/best.pt --img 260 --conf 0.5 --source ./Brain_tumor_YOLO/Testing/meningioma
Detect: weights[./runs/train/exp/weights/best.pt], source=./Brain_tumor_YOLO/Testing/meningioma, data=data/coco128.yaml, imgs=[260, 260], conf_thres=0.5, iou_thres=0.45, max_det=1000, device=,
view_img=False, save_txt=False, save_format=0, save_csv=False, save_conf=False, save_crop=False, nosave=False, classes=None, agnostic_nms=False, augment=False, visualize=False, update=False, pr
ject=runs/detect, name=exp, exist_ok=False, line_thickness=3, hide_labels=False, hide_conf=False, half=False, dnn=False, vid_stride=1
Fatal: cannot change to 'C:\Users\Sam\Sam\Desktop\Code\VS': No such file or directory
YOLOv5 2025-3-27 Python-3.12.7 torch-2.6.0+cpu CPU

Fusing layers...
Model summary: 212 layers, 20865057 parameters, 0 gradients, 47.9 GFLOPs
WARNING --img-size [260, 260] must be multiple of max stride 32, updating to [288, 288]
Image 1/10 C:\Users\Sam\Sam\Desktop\Code\VS CODE\VS5\py detect.py 413 Lab 4\Brain_tumor_YOLO\Testing\meningioma\Te-me_0025.jpg: 288x288 1 meningioma, 102.1ms
Image 2/10 C:\Users\Sam\Sam\Desktop\Code\VS CODE\VS5\py detect.py 413 Lab 4\Brain_tumor_YOLO\Testing\meningioma\Te-me_0083.jpg: 288x288 (no detections), 91.6ms
Image 3/10 C:\Users\Sam\Sam\Desktop\Code\VS CODE\VS5\py detect.py 413 Lab 4\Brain_tumor_YOLO\Testing\meningioma\Te-me_0092.jpg: 288x256 1 meningioma, 143.8ms
Image 4/10 C:\Users\Sam\Sam\Desktop\Code\VS CODE\VS5\py detect.py 413 Lab 4\Brain_tumor_YOLO\Testing\meningioma\Te-me_0097.jpg: 288x288 1 meningioma, 107.8ms
Image 5/10 C:\Users\Sam\Sam\Desktop\Code\VS CODE\VS5\py detect.py 413 Lab 4\Brain_tumor_YOLO\Testing\meningioma\Te-me_0126.jpg: 288x288 1 meningioma, 91.7ms
Image 6/10 C:\Users\Sam\Sam\Desktop\Code\VS CODE\VS5\py detect.py 413 Lab 4\Brain_tumor_YOLO\Testing\meningioma\Te-me_0175.jpg: 288x288 1 meningioma, 96.6ms
Image 7/10 C:\Users\Sam\Sam\Desktop\Code\VS CODE\VS5\py detect.py 413 Lab 4\Brain_tumor_YOLO\Testing\meningioma\Te-me_0211.jpg: 288x288 1 meningioma, 89.5ms
Image 8/10 C:\Users\Sam\Sam\Desktop\Code\VS CODE\VS5\py detect.py 413 Lab 4\Brain_tumor_YOLO\Testing\meningioma\Te-me_0218.jpg: 288x288 1 meningioma, 92.3ms
Image 9/10 C:\Users\Sam\Sam\Desktop\Code\VS CODE\VS5\py detect.py 413 Lab 4\Brain_tumor_YOLO\Testing\meningioma\Te-me_0215.jpg: 288x288 1 meningioma, 93.9ms
Image 10/10 C:\Users\Sam\Sam\Desktop\Code\VS CODE\VS5\py detect.py 413 Lab 4\Brain_tumor_YOLO\Testing\meningioma\Te-me_0304.jpg: 288x288 1 pituitary, 92.6ms
Speed: 0.2ms pre-process, 100.1ms inference, 0.5ms NMS per image at shape (1, 3, 288, 288)
Results saved to runs/detect/exp9
```

5.6 At the end of the test, the path to the annotated images by the trained algorithm will be provided to you. Choose one image of each class at your own consent and show the result of your model's annotation.

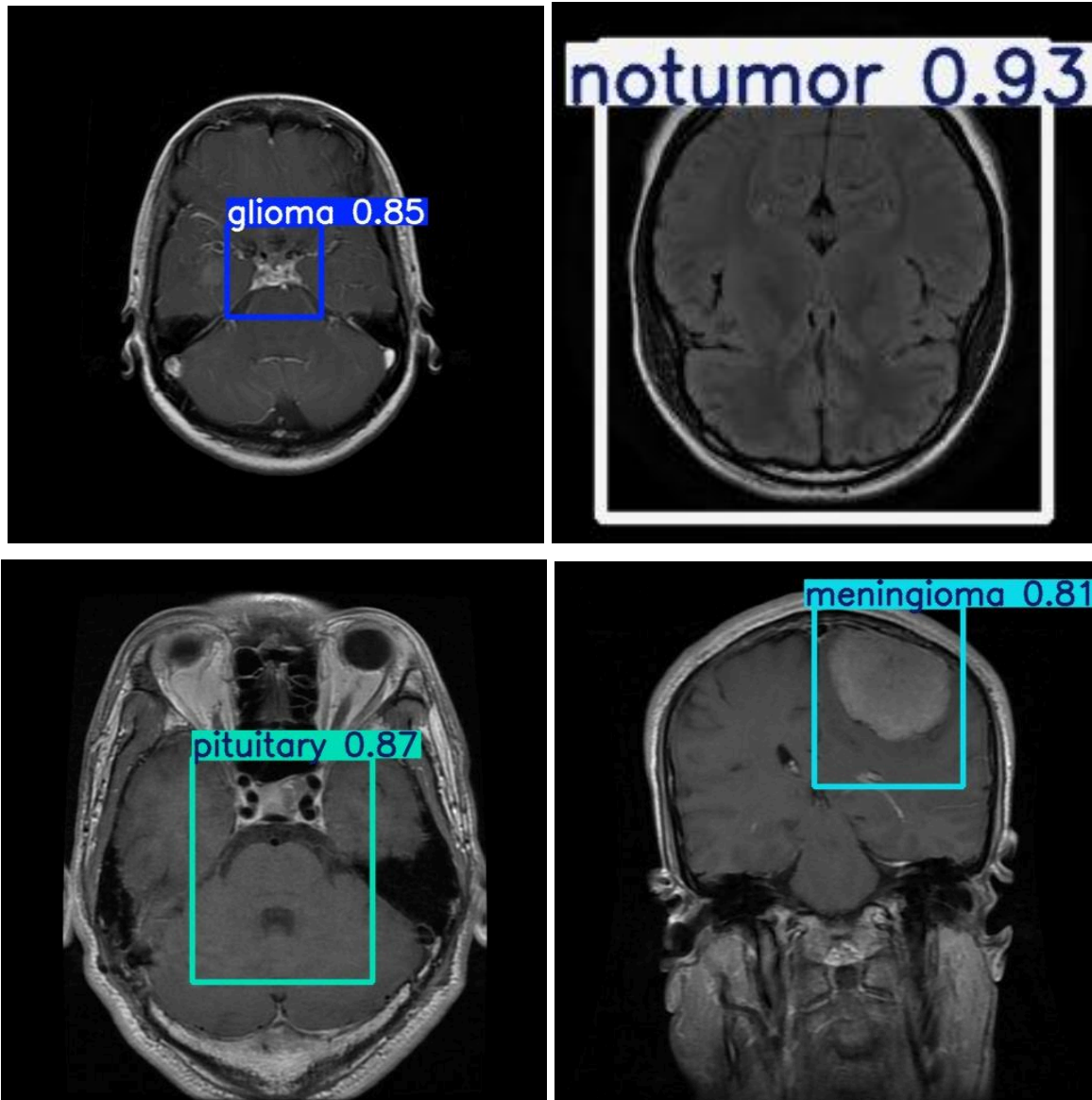


Figure 9: Final evaluation of each brain tumor detection case

The model can capture the abnormalities in the brain structure and separate the healthy brain from common brain tumors. There are many more brain tumors that this model does not cover. In the end, it still requires a doctor to examine the MRI brain scans themselves to make the call.